AYUSH — 3-Minute College Presentation Script

Territory Runner | Subsystem Focus: Backend, Concurrency & Spatial DB

TOTAL TIME: 3:00 MIN

ROLE: BACKEND LEAD

PRESENTER	ASSIGNED TIME	CORE STACK
AYUSH	Minute 0:00 to 3:00 (3 Mins)	Go 1.24, PostGIS 3.4, Advisory Locks

⏱️ MINUTE 1 (0:00 – 1:00) : Hook & Server-Authoritative Architecture

0:00 - 1:00

"Good morning respected professors and examiners. My name is **Ayush**. In *Territory Runner*, I architected the **server-authoritative backend, PostGIS spatial database, distributed concurrency control, and kinematic anti-cheat**.

Traditional fitness trackers like Strava only record passive 1D polylines without real-world conflict. Territory Runner transforms outdoor workouts into an active spatial strategy platform where athletes physically capture **arbitrary 2D closed-loop polygons**. Because mobile devices operate in untrusted environments susceptible to GPS spoofing and clock manipulation, our core architectural rule is: *the client proposes, but the server authoritatively validates, serializes, and commits*."

> 👉 ***Action:*** *Click Slide 1 & Slide 2. Point to the architecture diagram showing mobile requests flowing into the Chi REST middleware and PostgreSQL.*

⏱️ MINUTE 2 (1:00 – 2:00) : Spatial Concurrency & Kinematic Anti-Cheat

1:00 - 2:00

"The most complex engineering challenge was **spatial concurrency control**. When rival runners compete in the same park, multiple writes hit the same geographic polygons. Optimistic Concurrency Control fails because discarded PostGIS difference queries waste massive CPU. Redis Redlock introduces split-brain risk.

I solved this using **PostgreSQL transaction-scoped advisory locks** via `pg_advisory_xact_lock`. This serializes spatial mutations—`ST_Union` to claim territory and `ST_Difference` to carve rivals—with zero orphaned lock leaks.

Additionally, to prevent players from driving cars or using mock GPS apps, I authored a **kinematic anti-cheat filter** in Go. It enforces hard athletic velocity ceilings: **25 km/h for running** and **50 km/h for cycling**, with a max acceleration limit of **4.5 m/s²**, automatically rejecting vehicle spoofers."

> 👉 ***Action:*** *Switch to Slide 3. Highlight the Advisory Lock comparison table and the speed ceiling boundaries.*

⏱️ MINUTE 3 (2:00 – 3:00) : Idempotency, Verification & Handover

2:00 - 3:00

"When mobile clients submit runs in spotty network zones, retried requests can cause duplicate territory claims. I enforced strict **idempotency keys** in PostgreSQL: duplicate submissions return HTTP 200 with cached capture results without double-crediting XP or land.

To verify the backend, I authored **79 automated Go tests** covering advisory lock stress, kinematic limits, and spatial transactions. Under load, our p95 transaction latency remains under **35 milliseconds**, and the entire backend is containerized in Alpine Docker.

Now, I hand over to my teammate **Gurudutt**, who will present the mobile UI design and athletic design system."

> 👉 ***Action:*** *Switch to Slide 4. Show the 79 Go tests passing badge and nod toward Gurudutt for a smooth transition.*

AYUSH — Slide Deck Presentation Guide

Exactly what to display and point at during your 3 minutes

4 SLIDES TOTAL

SLIDE 1 (0:00 - 0:30) — INTRO & PROBLEM

The Distributed Spatial Challenge

- Strava passive 1D linear tracking vs Territory Runner 2D organic polygons.
- Untrusted mobile environments: GPS spoofing, clock skew, packet sniffing.
- **Architecture Rule:** Server-Authoritative Go 1.24 + PostGIS 3.4 engine.

SLIDE 2 (0:30 - 1:00) — BACKEND ARCHITECTURE

Go Chi REST & Middleware Pipeline

- Chi router with rate limiting, CORS whitelist, body limit guards.
- Firebase RS256 JWT public key verification with in-memory JWKS caching.
- Spatial Entity-Relationship schema: `users`, `territories`, `runs`.

SLIDE 3 (1:00 - 2:00) — LOCKS & ANTI-CHEAT

Advisory Locks & Kinematic Velocity Ceilings

- `pg_advisory_xact_lock(hashtext('territory_mutation_lock'))`.
- Zero deadlocks, 100% ACID compliance, auto-released on commit/rollback.
- Anti-Cheat Ceilings: Running: 25 km/h | Cycling: 50 km/h | Accel: 4.5 m/s².

SLIDE 4 (2:00 - 3:00) — INTEGRITY & METRICS

Idempotency & Verification Metrics

- Client UUID `idempotency_key` preventing double territory claims.
- PostGIS `ST_MakeValid` repairing bowtie self-intersections.
- **79 Go Tests Passing** | p95 Latency < 35ms | Minimal Alpine Docker container.

📋 Ayush's Quick Reference Pocket Card

CODE & FILES

- **Server Entry:** `backend/cmd/server/main.go`
- **Spatial DB Store:** `backend/internal/store/postgres.go` (`ST_Union`, `ST_Difference`, `ST_MakeValid`)
- **Advisory Locking:** `backend/internal/store/advisory_lock.go`
- **Anti-Cheat Engine:** `backend/internal/anticheat/kinematic.go`
- **Key SQL:** `SELECT pg_advisory_xact_lock(hashtext('territory_mutation_lock'));`

AYUSH — Examiner Viva Defense Cheat Sheet

Top 4 killer examiner questions with rapid 15-second oral defenses

EXAMINER DEFENSE

Q1: Why use PostgreSQL advisory locks (`pg_advisory_xact_lock`) instead of Redis Redlock or row locks?

15-Second Defense: "Row locks cannot lock rows that don't exist yet—like newly carved rival territory slices—and table locks freeze read queries. Redis Redlock requires managing dual state and risks split-brain errors under clock skew. `pg_advisory_xact_lock` executes inside the PostgreSQL kernel, serializes spatial writes cleanly, and automatically releases on commit or rollback."

Implemented in: `backend/internal/store/postgres.go#L180-L215`

Q2: How does your kinematic filter avoid falsely flagging an athletic human sprinter?

15-Second Defense: "We use world-record human biomechanics. Sprinters top out at ~44 km/h for brief bursts but cannot sustain it, and humans exhibit natural acceleration-deceleration curves. Our running ceiling is 25 km/h sustained with a 4.5 m/s^2 acceleration ceiling over a sliding time window. Vehicles show prolonged high velocities across turns that easily trigger our 0.70 anomaly threshold."

Implemented in: `backend/internal/anticheat/kinematic.go#L45-L95`

Q3: How does PostGIS `ST_MakeValid` handle self-intersecting 'bowtie' loops from GPS jitter?

15-Second Defense: "In OGC geometry, self-intersecting loops break simple feature rules and throw GEOS TopologyExceptions. We pass incoming WKT through `ST_Multi(ST_CollectionExtract(ST_MakeValid(geom), 3))`. Under the hood, GEOS splits bowtie self-crossings into valid multi-polygon sub-rings, and collection extract preserves only 2D polygons while stripping 1D antenna lines."

Implemented in: `backend/internal/store/postgres.go#L240-L260`

Q4: How do you verify Firebase JWT tokens on the Go backend without calling Google on every request?

15-Second Defense: "We fetch Google's public JWKS certificates once on server startup and cache them in memory. Every incoming request is verified in-process using local asymmetric RS256 RSA public key cryptography. We only make a network call to refresh certificates if an unknown key ID (`kid`) is encountered."

Implemented in: `backend/internal/auth/verifier.go#L30-L75`

Candidate Signature	Internal Project Guide	External Viva Examiner
AYUSH	Dept. of CSE	University Board